

SimpliCulinary

Rapport Complet de Configuration du Projet Django

Atelier 3 & 4 — Environnement · Backend · Templates · Fichiers Statiques

Atelier 3

Atelier 4

Système de templates Bootstrap, icônes
FontAwesome, héritage des templates,
gestion des URLs

Statut global : **Configuration terminée avec succès ✓**
Toutes les phases des ateliers 3 et 4 ont été complétées.

ATELIER 3 — Configuration Initiale du Projet Django

I Configuration de l'Environnement Virtuel

L'objectif de cette phase est de créer un espace isolé pour gérer les dépendances du projet, garantissant que les paquets installés ne conflictuent pas avec d'autres projets Python présents sur la machine.

1.1 — Initialisation du Répertoire

Un répertoire dédié au projet a été créé et ouvert dans l'IDE Visual Studio Code. Cette étape constitue le point de départ de toute la structure du projet.

- Créer un dossier pour le projet
- Ouvrir le dossier dans VSCode

1.2 — Installation de Virtualenv

La bibliothèque virtualenv a été installée globalement via pip :

```
pip install virtualenv
```

1.3 — Création et Activation de l'Environnement

Génération de l'environnement virtuel :

```
virtualenv venv
```

Sur Windows (PowerShell), autorisation des scripts d'activation :

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

Activation de l'environnement virtuel :

```
myenv\scripts\activate
```

- Une fois activé, le nom de l'environnement apparaît entre parenthèses dans le terminal, indiquant que toutes les commandes pip opèrent dans l'espace isolé.

II Création du Backend Django

Cette phase couvre l'installation du framework Django et l'initialisation de la structure principale du serveur web.

2.1 — Installation de Django

Dans l'environnement virtuel activé, Django a été installé via pip :

```
pip install django
```

2.2 — Initialisation du Projet

Le projet principal nommé « backend » a été créé via l'outil d'administration Django :

```
django-admin startproject backend
```

Cette commande génère automatiquement la structure de fichiers standard : settings.py, urls.py, wsgi.py, asgi.py, et manage.py.

2.3 — Vérification du Serveur de Développement

Le bon fonctionnement a été vérifié en lançant le serveur de développement :

```
cd backend
python manage.py runserver
```

■ La page d'accueil par défaut de Django a été affichée avec succès sur `http://127.0.0.1:8000`, confirmant que l'installation est entièrement opérationnelle.

III Structure Modulaire des Applications

Le projet SimpliCulinary adopte une architecture modulaire en séparant les fonctionnalités en plusieurs applications Django indépendantes. Cette approche facilite la maintenabilité, la réutilisabilité du code et le travail collaboratif en équipe.

3.1 — Création des Applications

Chaque module a été généré avec la commande Django dédiée :

```
python manage.py startapp [nom_app]
```

Application	Description	Statut
products	Catalogue des produits culinaires	✓ Créée
users	Gestion des utilisateurs et authentification	✓ Créée
cart	Gestion du panier d'achat	✓ Créée
orders	Traitement et suivi des commandes	✓ Créée
base	App centrale — templates Bootstrap, navigation	✓ Créée

3.2 — Déclaration dans settings.py

Chaque application créée a été déclarée dans la liste `INSTALLED_APPS` du fichier `backend/backend/settings.py` pour être reconnue par le framework Django :

```
INSTALLED_APPS = [
    ...
    'products',
    'users',
    'cart',
    'orders',
    'base',
]
```

Récapitulatif — Atelier 3

Phase	Statut
I — Environnement Virtuel	✓ Terminé

II — Backend Django	✓ Terminé
III — Structure Modulaire (5 apps)	✓ Terminé

ATELIER 4 — Système de Templates Django avec Bootstrap & FontAwesome

I Configuration des Fichiers Statiques

Cette phase établit la structure des ressources statiques (CSS, JavaScript, icônes) et configure Django pour les servir correctement pendant le développement.

1.1 — Arborescence des Dossiers

À la racine du projet, la structure suivante a été créée :

Chemin	Contenu
<code>static/base/css/</code>	Fichiers CSS (Bootstrap, thème Bootswatch)
<code>static/base/js/</code>	Fichiers JavaScript (Bootstrap bundle)
<code>static/base/fontawesome/</code>	Dossiers <code>css/</code> et <code>webfonts/</code> de FontAwesome

1.2 — Installation de Bootstrap

Après téléchargement de Bootstrap (ou d'un thème Bootswatch), les fichiers ont été placés :

- CSS : `bootstrap.min.css` → `static/base/css/`
- JS : `bootstrap.bundle.min.js` → `static/base/js/`

1.3 — Installation de FontAwesome

Après téléchargement et extraction de FontAwesome, les dossiers suivants ont été copiés :

- `fontawesome/css/` → `static/base/fontawesome/css/`
- `fontawesome/webfonts/` → `static/base/fontawesome/webfonts/`

1.4 — Paramétrage Django (settings.py)

Deux paramètres ont été configurés dans `backend/backend/settings.py` :

```
STATIC_URL = 'static/'
STATICFILES_DIRS = [
    BASE_DIR / 'static',
]
```

- `STATICFILES_DIRS` indique à Django où chercher les fichiers statiques supplémentaires en dehors des dossiers `static/` internes à chaque application.

II Création du Template de Base (`base.html`)

Le fichier `templates/base/base.html` constitue le squelette commun à toutes les pages de l'application. Il intègre les ressources partagées et définit la structure HTML globale.

2.1 — En-tête HTML (<head>)

La directive de chargement des statiques et les liens CSS sont déclarés en premier :

```
{% load static %}
<link rel='stylesheet' href="{% static 'base/css/bootstrap.min.css' %}">
<link rel='stylesheet' href="{% static 'base/fontawesome/css/all.min.css' %}">
<title>{% block title %}{% endblock %}</title>
```

2.2 — Barre de Navigation Bootstrap

Une barre de navigation responsive (thème Flatly de Bootswatch) a été intégrée directement après la balise `<body>`. Elle comprend les liens requis :

Lien	Balise Django	Icône FontAwesome
Accueil	<code>{% url 'home' %}</code>	—
Nos produits	<code>{% url 'products_list' %}</code>	—
Login	<code>{% url 'login' %}</code>	<code>fa-solid fa-user</code>
Cart	<code>{% url 'cart' %}</code>	<code>fa-solid fa-cart-shopping</code>

2.3 — Zone de Contenu et Script JavaScript

Le bloc de contenu dynamique et le script Bootstrap sont positionnés :

```
{% block content %}{% endblock %}
<script src="{% static 'base/js/bootstrap.bundle.min.js' %}"></script>
```

III Héritage des Templates

Le système d'héritage Django permet à chaque page de réutiliser la structure commune définie dans `base.html`, en ne redéfinissant que le contenu spécifique à chaque vue.

3.1 — Exemple : `product_list.html`

Structure type d'un template héritant de `base.html` :

```
{% extends 'base/base.html' %}
{% block title %}Produits{% endblock %}
{% block content %}
<div class='container my-5'>
<!-- Contenu spécifique à la liste de produits -->
</div>
{% endblock %}
```

- La directive `{% extends %}` doit toujours être la première ligne du fichier template. Les blocs `{% block %}` peuvent être surchargés ou laissés vides pour conserver le contenu par défaut.

IV Gestion des URLs dans les Templates

La balise `{% url %}` génère dynamiquement les URLs en se basant sur les noms définis dans les fichiers `urls.py`, évitant ainsi les liens codés en dur.

Balise URL	Description	Statut
<code>{% url 'home' %}</code>	Lien vers la page d'accueil	✓ Configuré
<code>{% url 'products_list' %}</code>	Lien vers le catalogue de produits	✓ Configuré
<code>{% url 'login' %}</code>	Lien vers la page de connexion	✓ Configuré
<code>{% url 'cart' %}</code>	Lien vers le panier d'achat	✓ Configuré

Récapitulatif Général — Ateliers 3 & 4

L'ensemble des deux ateliers a été mené à terme avec succès. Le projet SimpliCulinary dispose désormais d'une architecture complète : backend Django modulaire, système de templates Bootstrap responsive, icônes FontAwesome, et gestion dynamique des URLs.

Atelier	Phase	Statut
Atelier 3	I — Environnement Virtuel	✓ Terminé
	II — Backend Django	✓ Terminé
	III — Structure Modulaire (5 apps)	✓ Terminé
Atelier 4	1 — Configuration des fichiers statiques	✓ Terminé
	2 — Template de base (base.html)	✓ Terminé
	3 — Héritage des templates	✓ Terminé
	4 — Gestion dynamique des URLs	✓ Terminé

Le projet est désormais prêt pour la prochaine étape : la conception des modèles de données Django (ORM), la création des vues, et le développement des fonctionnalités métier de SimpliCulinary.